
Quantization

INTRODUCTION TO LLM
INFERENCE SERVING SYSTEMS
CHUHONG YUAN





Quantization – What?

- Use data with lower precisions
- In LLM inference:
 - Weight
 - Activation: intermediate values, e.g., input embeddings, attention outputs, FFN outputs
 - KV Cache



Quantization – Why?

- Discussion



Quantization – Why?

- Discussion
- Improve throughput
- Save memory footprints



Quantization – How?

- $Q_X = \left\lfloor \frac{X}{s} + z \right\rfloor, s = \frac{X_{max} - X_{min}}{q_{max} - q_{min}}, z = \left\lfloor q_{min} - \frac{X_{min}}{s} \right\rfloor$
- $\hat{X} = (Q_X - z) \cdot s$

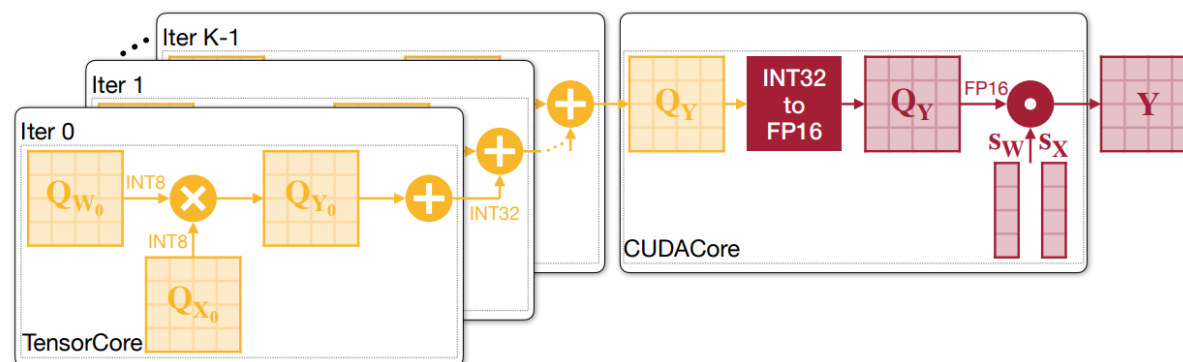
Quantization – How?

- $Q_X = \left\lfloor \frac{X}{s} + z \right\rfloor, s = \frac{X_{max} - X_{min}}{q_{max} - q_{min}}, z = \left\lfloor q_{min} - \frac{X_{min}}{s} \right\rfloor$
- $\hat{X} = (Q_X - z) \cdot s$
- Granularity
 - Per-tensor
 - Per-channel
 - Per-group

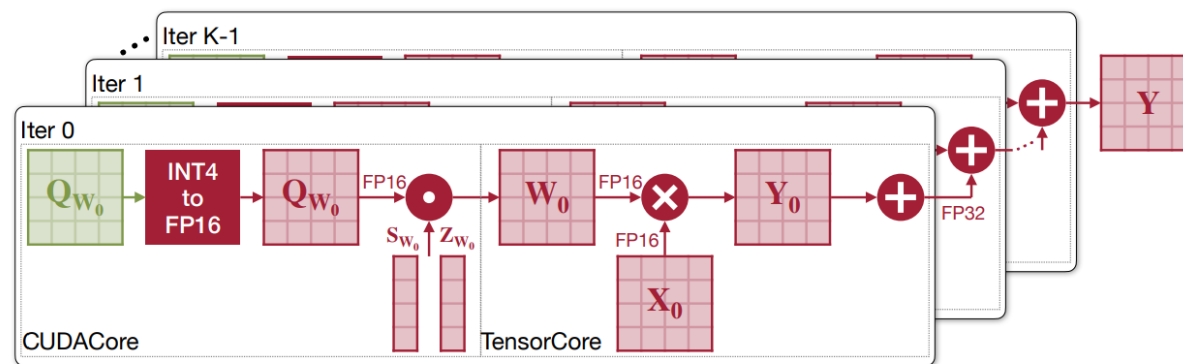
QServe



Motivation – W8A8, W4A16

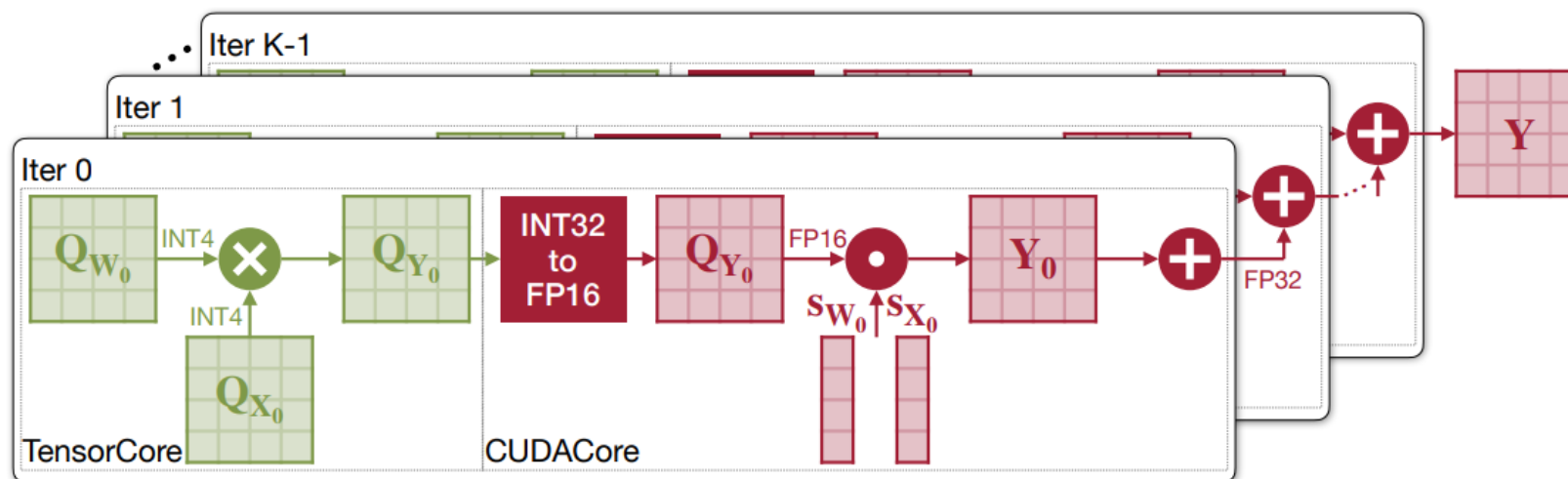


(a) TensorRT-LLM (INT8 Weights and INT8 Activations)



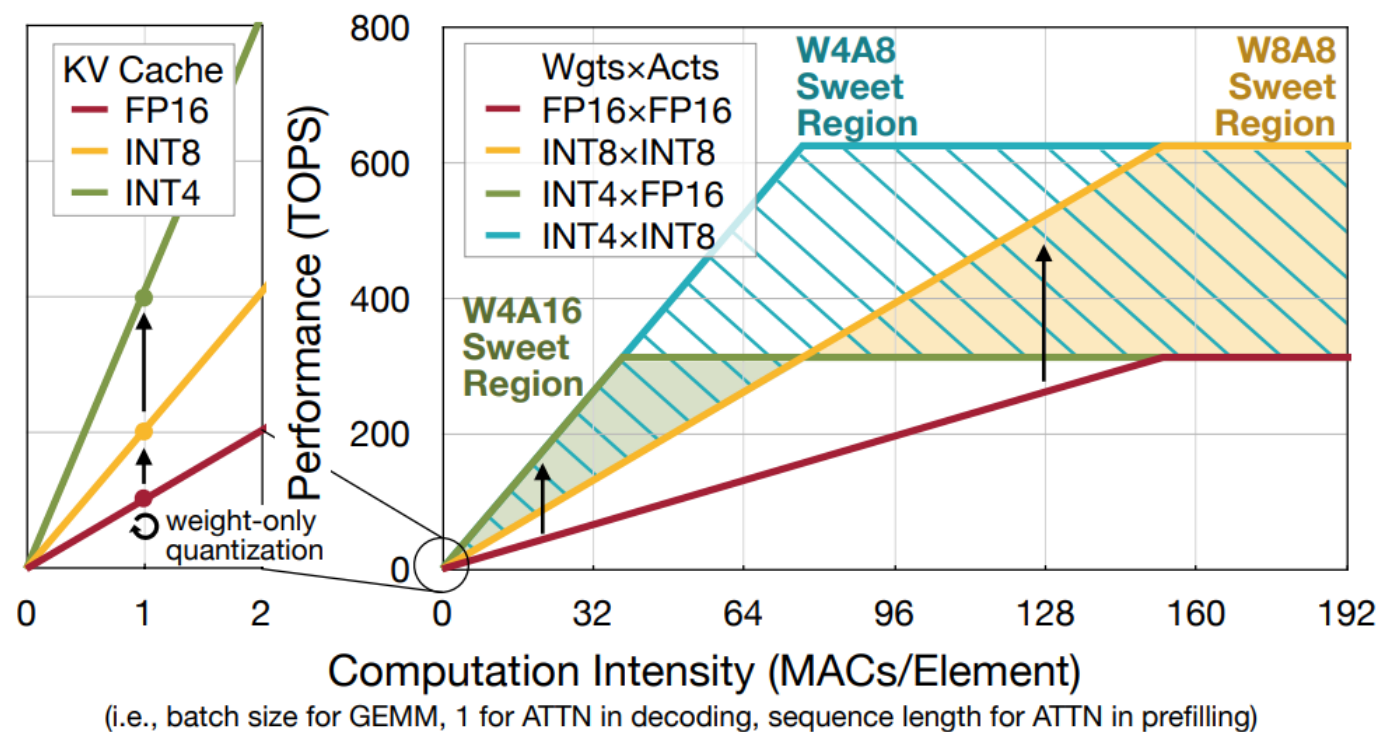
(b) TensorRT-LLM (INT4 Weights and FP16 Activations)

Motivation – W4A4

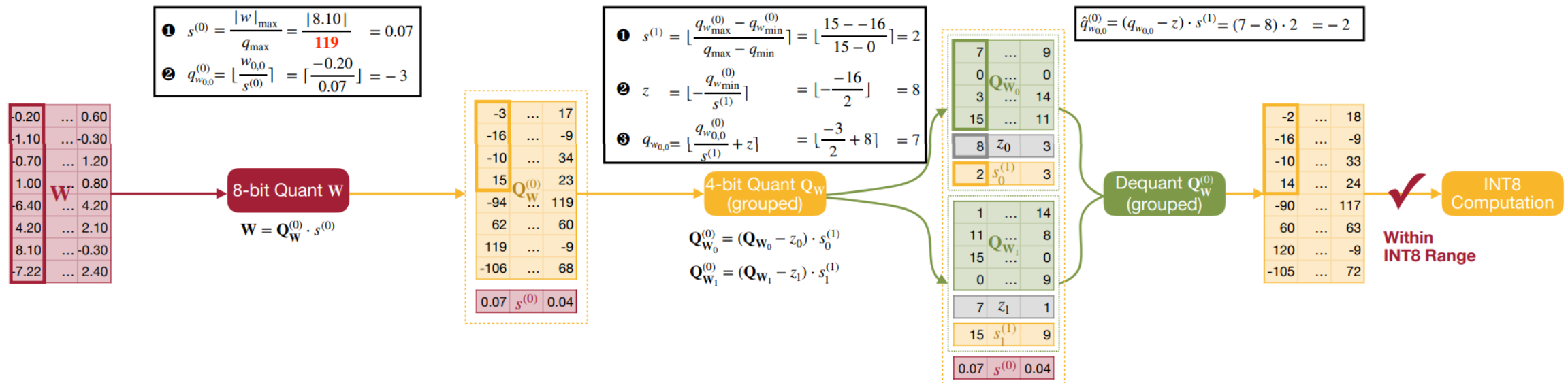


(c) ATOM (INT4 Weights and INT4 Activations)

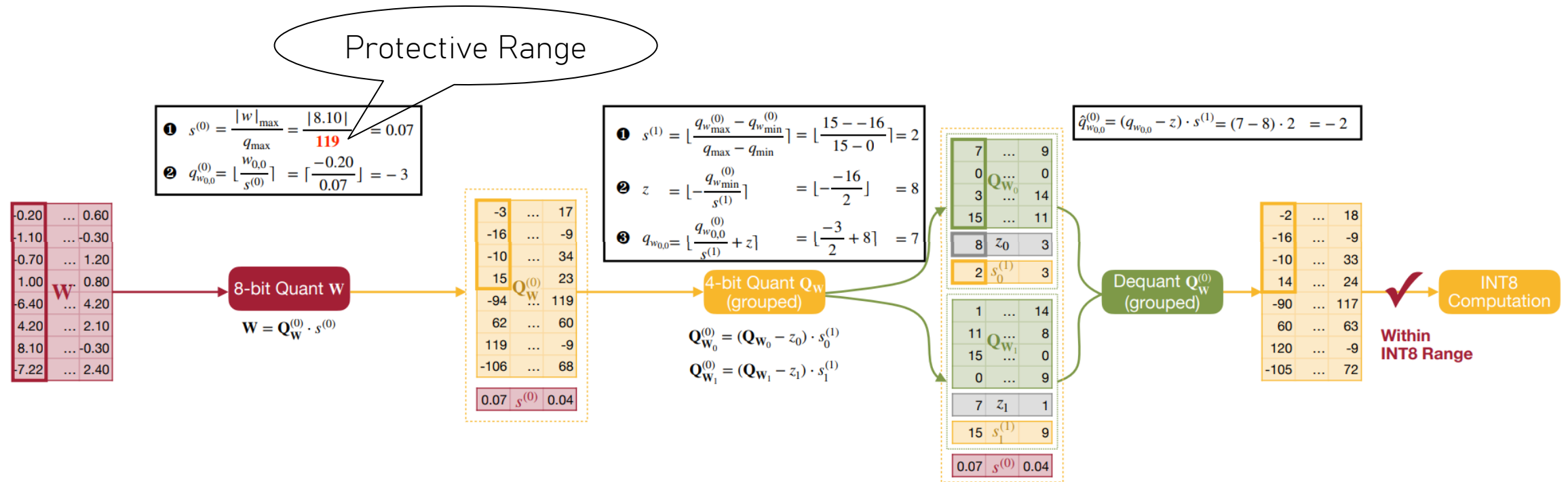
Motivation – Performance of Quantization



Progressive Group Quantization



Progressive Group Quantization

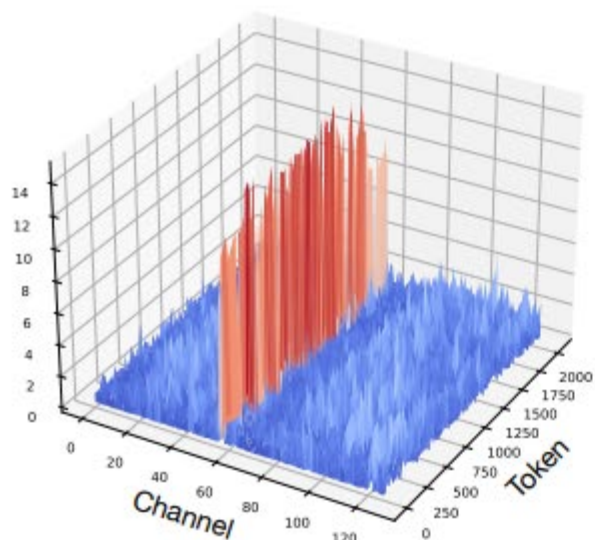




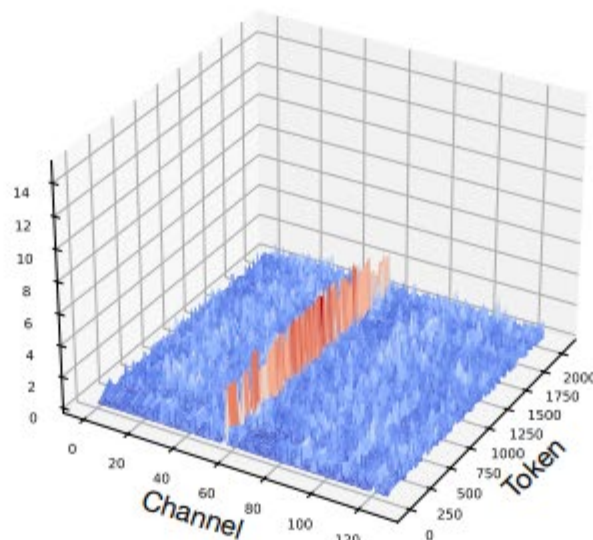
PGQ Advantages

- Less computation in CUDA cores
- Able to dequantize to INT8, comparing to VSQuant and DoubleQuant
- DGQ separates the dequantization kernel with the GEMM kernel

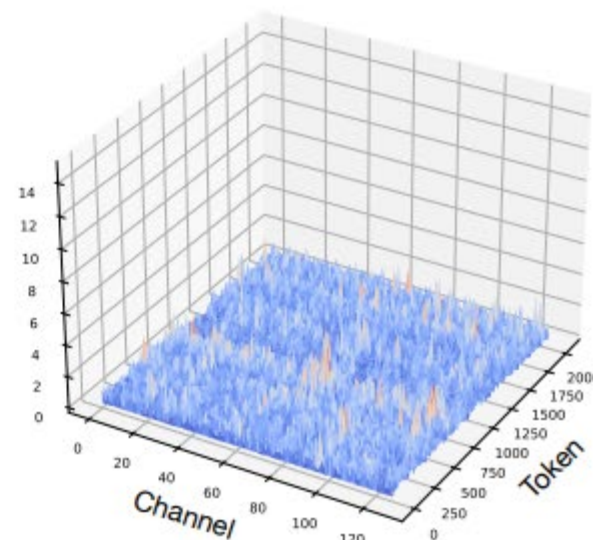
SmoothAttention



Layer 24 post-RoPE Keys
(Original)



Layer 24 post-RoPE Keys
(SmoothAttention)



Layer 24 Values

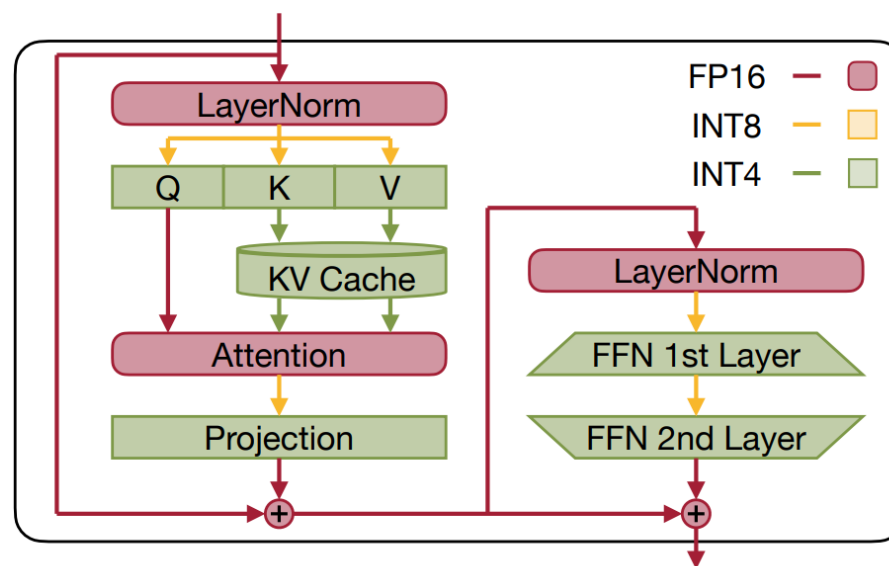
Scale down
by λ



General Optimizations

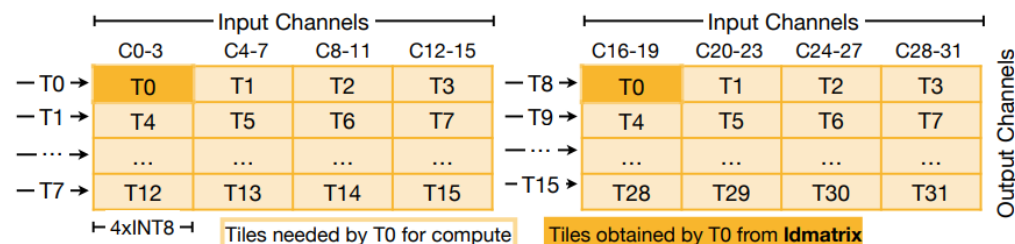
- Block input module rotation
- Block output module smoothing
- Activation-aware channel reordering
- Weight clipping

QServe System

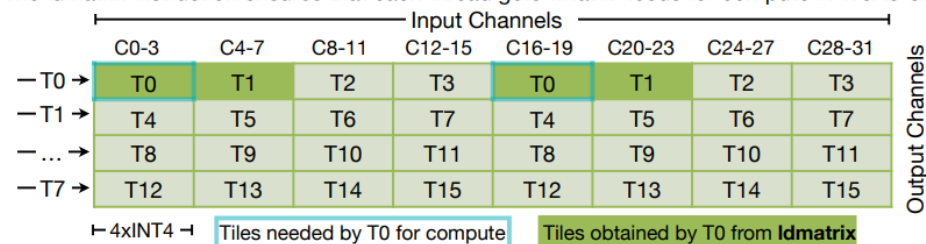


*Figure 11. QServe's precision mapping for an FP16 in, FP16 out LLM block. All GEMM operators take in **W4A8** inputs and produce FP16 outputs. Activation quantization happens in normalization and activation layers.*

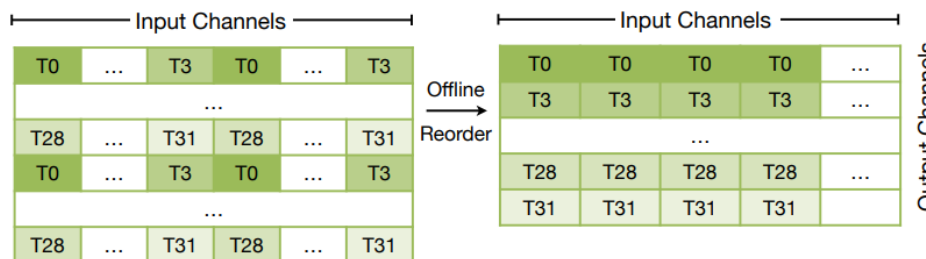
W4A8 GEMM



(a) The Idmatrix instruction ensures that each thread gets what it needs for compute in W8A8 GEMM



(b) However, the Idmatrix instruction fails for W4A8 GEMM due to **storage-compute mismatch**



(c) Our solution: **compute-aware weight reorder**

Fast Dequantization



Figure 13. QServe exploits **register-level parallelism** to significantly reduce the number of required logical operations in `UINT4` to `UINT8` weight unpacking.

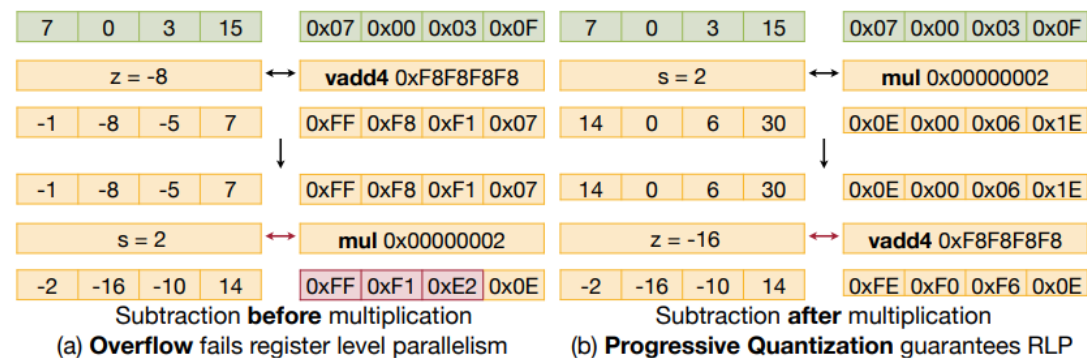


Figure 14. Our progressive quantization algorithm ensures that all intermediate results in the **subtraction after multiplication** computation order will not overflow, thereby enabling **register-level parallelism** and reducing main loop overhead.



Evaluation – Settings

- GPUs: L40S 48G, A100 80G
- Baseline system: TensorRT-LLM
- Models: Llama, Mistral, Yi
- Efficiency workload: input 1024, output 512

Evaluation – Accuracy

Table 2. WikiText2 perplexity with 2048 sequence length. The lower is the better.

WikiText2 Perplexity ↓		Llama-3		Llama-2		Llama			Mistral	Mixtral	Yi
Precision	Algorithm	8B	7B	13B	70B	7B	13B	30B	7B	8x7B	34B
FP16	-	6.14	5.47	4.88	3.32	5.68	5.09	4.10	5.25	3.84	4.60
W8A8	SmoothQuant	6.28	5.54	4.95	3.36	5.73	5.13	4.23	5.29	3.89	4.69
W4A16 g128	GPTQ-R	6.56	5.63	4.99	3.43	5.83	5.20	4.22	5.39	4.08	4.68
	AWQ	6.54	5.60	4.97	3.41	5.78	5.19	4.21	5.37	4.02	4.67
W4A4	QuaRot	8.20	6.10	5.40	3.79	6.26	5.55	4.60	5.71	NaN	NaN
		8.33	6.19	5.45	3.83	6.34	5.58	4.64	5.77	NaN	NaN
W4A4 g128	QuaRot [†]	7.32	5.93	5.26	3.61	6.06	5.40	4.44	5.54	NaN	NaN
		7.51	6.00	5.31	3.64	6.13	5.43	4.48	5.58	NaN	NaN
	Atom [†]	7.57	6.03	5.27	3.69	6.16	5.46	4.55	5.66	4.42	4.92
		7.76	6.12	5.31	3.73	6.25	5.52	4.61	5.76	4.48	4.97
W4A8KV4	RTN	9.50	6.51	5.40	3.90	6.51	5.71	4.91	6.18	5.02	6.52
	AWQ	7.90	6.28	5.25	3.68	6.33	5.59	4.61	5.92	4.58	5.26
	QoQ	6.81	5.75	5.11	3.50	5.92	5.27	4.31	5.44	4.17	4.73
W4A8KV4 g128	RTN	7.25	5.99	5.19	3.70	6.23	5.46	4.56	5.59	4.39	5.49
	AWQ	6.94	5.83	5.12	3.51	5.93	5.36	4.39	5.50	4.23	4.78
	QoQ	6.70	5.67	5.06	3.46	5.88	5.23	4.27	5.41	4.13	4.73

* Grayed results use Wikitext2 as calibration dataset.

† QuaRot and Atom apply group quantization to activations as well.

Evaluation – Efficiency

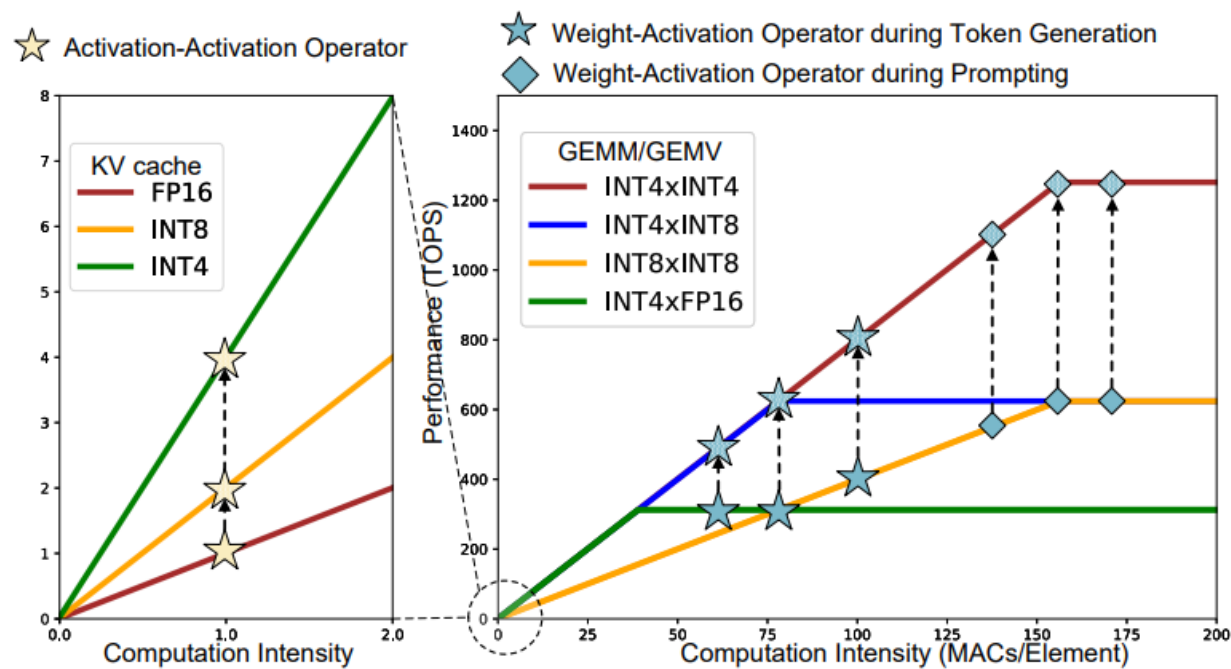
Table 4. The absolute token generation throughput of QServe and TensorRT-LLM in Fig. 15. *: we calculate the speedup over highest achievable throughput from TensorRT-LLM across all three precision configurations. Our QServe system achieves competitive throughput on L40S GPU compared to TensorRT-LLM on A100, effectively reducing the dollar cost of LLM serving by 3 $\times$. Unit: tokens/second.

Device	System	Llama-3 8B	Llama-2 7B	Mistral 7B	LLama-2 13B	LLaMA 30B	Yi 34B	Llama-2 70B	Qwen1.5 72B
L40S	TRT-LLM-FP16	1326	444	1566	92	OOM	OOM	OOM	OOM
	TRT-LLM-W4A16	1431	681	1457	368	148	313	119	17
	TRT-LLM-W8A8	2634	1271	2569	440	123	364	OOM	OOM
	QServe (Ours)	3656	2394	3774	1327	504	869	286	59
	Speedup*	1.39$\times$	1.88$\times$	1.47$\times$	3.02$\times$	3.41$\times$	2.39$\times$	2.40$\times$	3.47$\times$
A100	TRT-LLM-FP16	2503	1549	2371	488	80	145	OOM	OOM
	TRT-LLM-W4A16	2370	1549	2403	871	352	569	358	143
	TRT-LLM-W8A8	2396	2334	2427	1277	361	649	234	53
	QServe (Ours)	3005	2908	2970	1741	749	797	419	340
	Speedup*	1.20$\times$	1.25$\times$	1.22$\times$	1.36$\times$	2.07$\times$	1.23$\times$	1.17$\times$	2.38$\times$

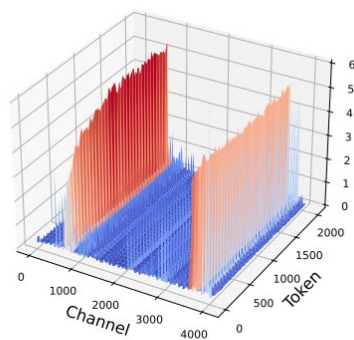
COMET



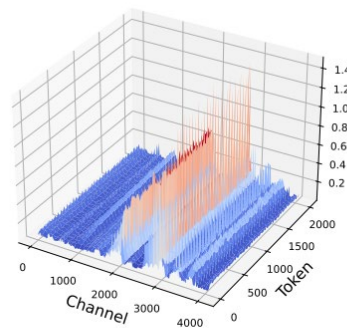
Motivation



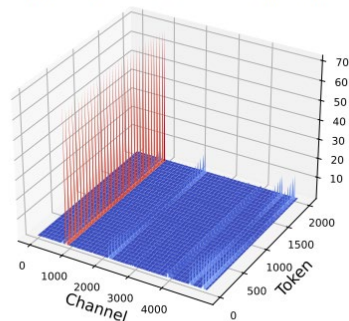
Activation Distribution



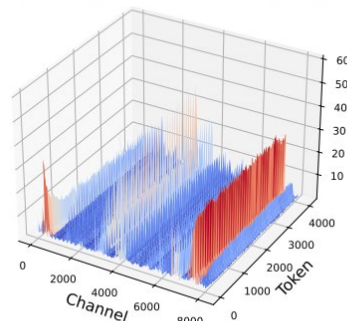
(a) Layer10_Attn_K_proj



(b) Layer29_Attn_O_proj



(c) Layer2_Attn_Q_proj



(d) Layer35_Attn_K_proj

FMPQ

- Fine-Grained Mixed-Precision

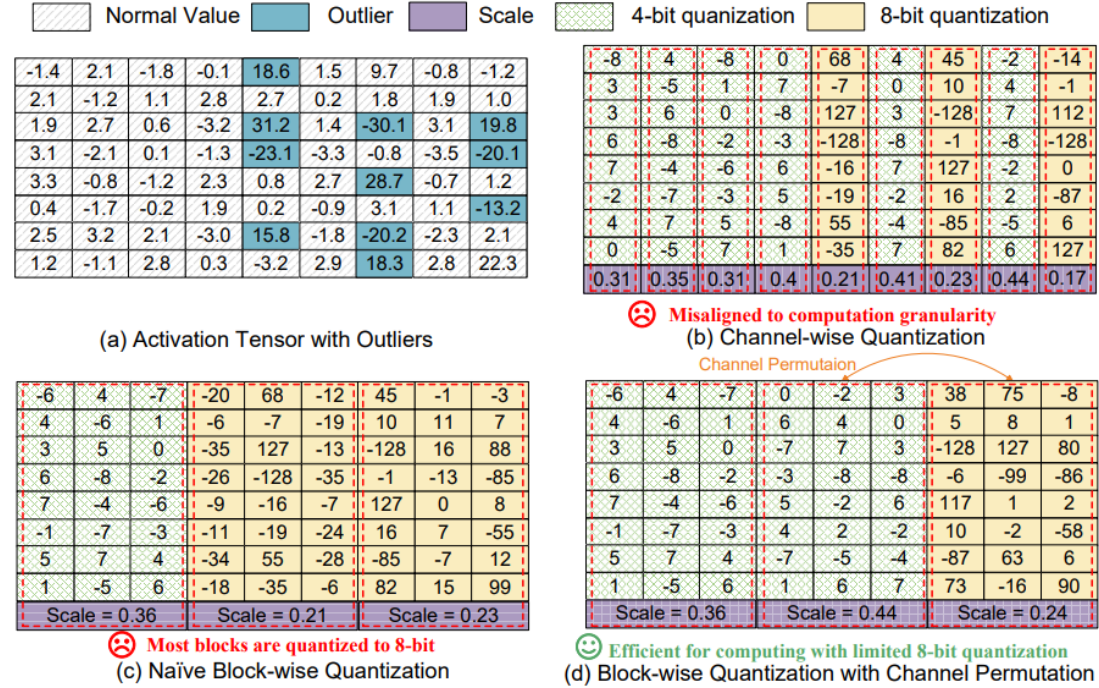
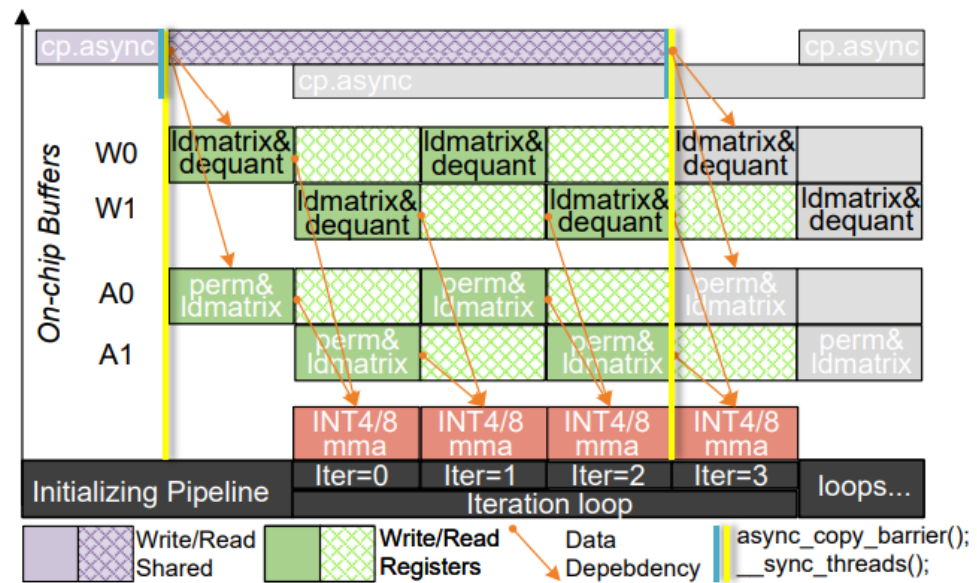


Figure 4. The algorithm design of FMPQ.

Pipeline



(c) SIMT-enhanced software pipeline.

Data Management

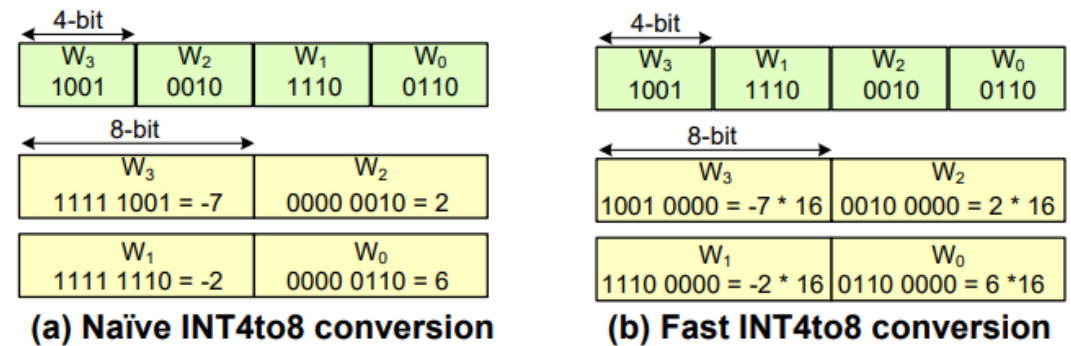
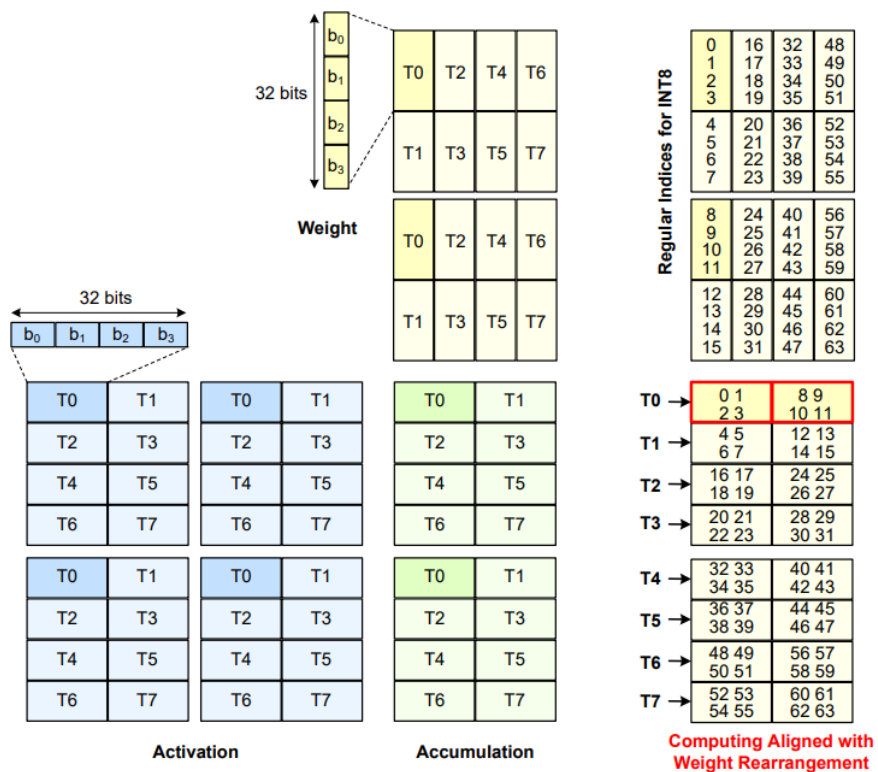
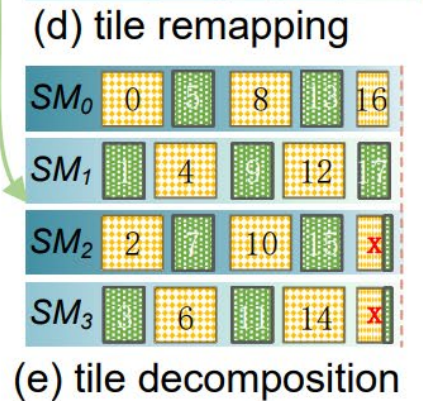
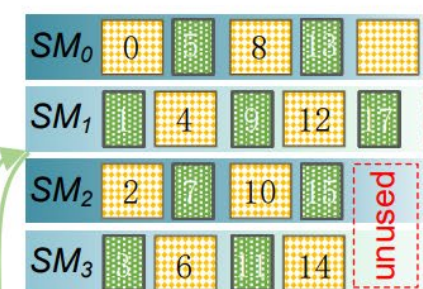
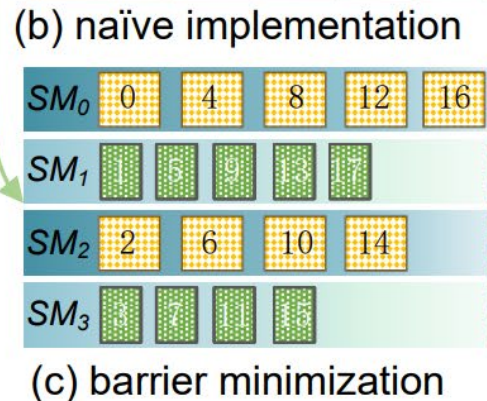
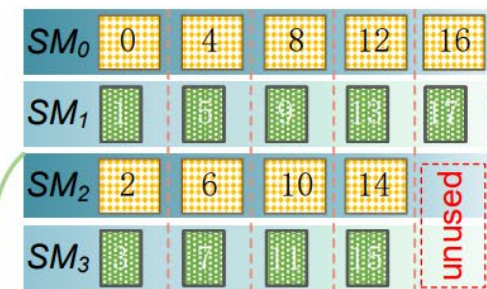
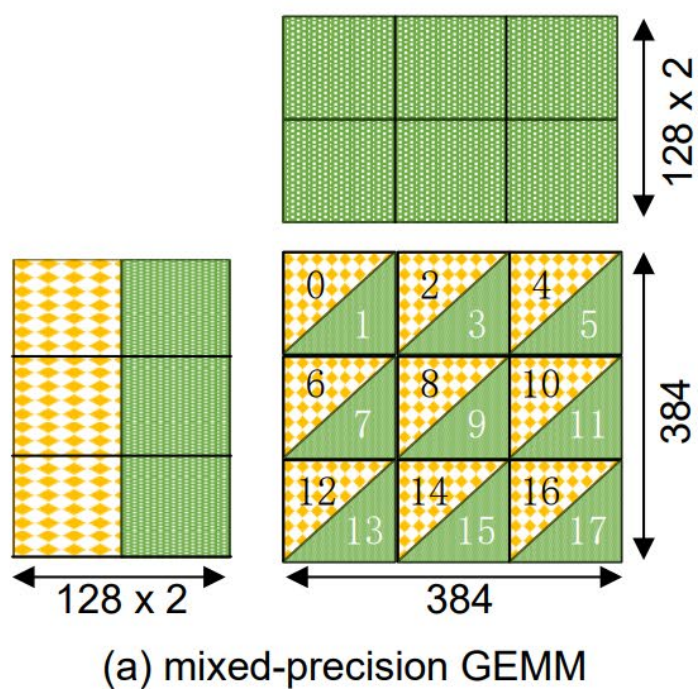


Figure 7. Fast data conversion from INT4 to INT8.

SM Scheduling





Evaluation – Settings

- GPU A100 80GB
- TensorRT-LLM v0.10.0
- Models: Llama, Mistral, OPT, Qwen

Evaluation – Accuracy

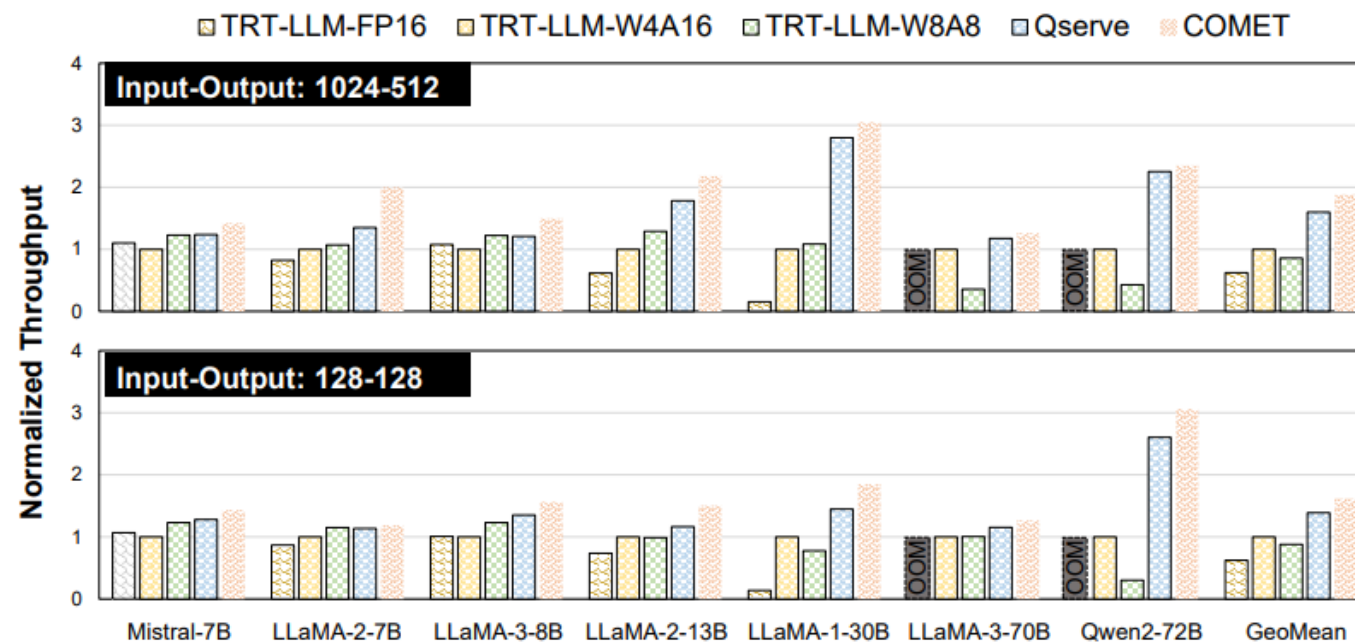
Table 1. Evaluation of WikiText2 perplexity for various quantized LLMs. Lower values indicate better performance.

Precision	Method	LLaMA-1			LLaMA-2			LLaMA-3		Mistral	OPT	Qwen2
		13B	30B	65B	7B	13B	70B	8B	70B	7B	13B	72B
FP16	-	5.09	4.10	3.56	5.12	4.57	3.12	6.14	2.86	5.25	10.13	4.94
W8A8	SmoothQuant	5.19	4.23	3.75	5.54	4.95	3.36	6.28	2.99	5.29	10.47	5.14
W4A16	GPTQ	5.40	4.48	3.83	5.83	5.13	3.58	7.02	3.44	5.39	10.31	5.09
	AWQ	5.34	4.39	3.76	6.15	5.12	3.54	7.09	3.40	5.37	10.39	5.07
	Omniquant	5.21	4.25	3.71	5.74	5.02	3.47	6.81	3.29	5.31	10.30	5.03
W4Ax	FMPQ	5.29	4.27	3.78	5.71	5.10	3.48	6.88	3.36	5.35	10.34	5.05
W4A4	Omniquant	10.87	10.33	9.17	14.26	12.30	9.93	14.27	9.75	7.87	11.65	7.91
W4A8 KV4	QoQ	5.28	4.34	3.83	5.75	5.12	3.52	6.89	3.38	5.45	10.42	5.21
W4AxKV4	FMPQ	5.32	4.31	3.82	5.73	5.19	3.56	6.91	3.41	5.43	10.44	5.17

Table 2. Zero-shot accuracy evaluation on five common sense tasks for LLaMA-3 family models.

Size	#Configuration	Method	Zero-shot Accuracy ↑					
			PIQA	ARC-e	ARC-c	HellaSwag	Winogrande	Avg.
8B	FP16	Full Precision	79.9	80.1	50.4	60.2	72.8	68.6
	W8A8	SmoothQuant	79.5	79.7	49.0	60.0	73.2	68.3
	W4A16	Omniquant	78.4	77.9	48.5	58.8	72.7	67.2
	W4A8 KV4	QoQ	77.1	77.2	47.8	57.6	72.0	66.3
	W4AxKV4	FMPQ	77.5	76.7	47.5	58.9	72.1	66.5
70B	FP16	Full Precision	82.4	86.9	60.3	66.4	80.6	75.3
	W8A8	SmoothQuant	82.2	86.9	60.2	66.3	80.7	75.3
	W4A16	Omniquant	82.7	86.3	59.0	65.7	80.9	74.9
	W4A8 KV4	QoQ	81.4	85.7	58.4	64.9	79.9	74.0
	W4AxKV4	FMPQ	82.5	85.2	58.3	65.0	79.6	74.1

Evaluation - Efficiency





Homework Late Submission Policy

- Submission due extended for 1 hour
- **May** raise the threshold by 1, but it is conditional – need to submit previously unsubmitted ones
- If the policy applies, resubmissions will be open in mid-November



Homework

- Read the related materials, and answer the following questions:
 - 1. LServe proposes a unified sparse attention framework. How does it work for the prefill and decoding phase, respectively?
 - 2. What are the two sparse patterns that substantially contribute to the attention score, according to SampleAttention? What are their semantics?
 - 3. How is CRA defined in SampleAttention? Why is it an indicator of the accuracy? How is it approximately determined?
- Related Materials:
 - LServe paper: <https://arxiv.org/pdf/2502.14866>
 - SampleAttention paper: <https://arxiv.org/pdf/2406.15486>

Q & A

